

S01 — Représenter des données (version élève)

Contents

S01 — Représenter des données : bits, textes et types construits	3
Situation-problème	3
Objectifs	3
Prérequis	3
Limites de la séquence	4
Formalisation	4
Définitions	4
Bases 2, 10 et 16	4
Entiers positifs et intervalle représentable	6
Entiers relatifs et complément à deux	6
Booléens et tables de vérité	7
Texte, ASCII et Unicode	8
Listes et tableaux Python	9
Tuples et p-uplets	9
Dictionnaires	10
Choisir une représentation	11
Tests et cas limites	11
Erreurs fréquentes — Récapitulatif	12
À retenir	12
Auto-évaluation	12
Trace écrite - S01	14
Notions essentielles	14
Définitions	14
Méthodes	14
Exemples minimaux	14
Points de vigilance	15
À savoir refaire	15
Auto-positionnement	15
TD - S01 Représentation des données	16
Situation-problème	16
Objectifs	16
Exercices	16
Aides progressives	17
Erreurs fréquentes	17

Auto-évaluation	18
Corrigé associé	18
Écriture de code	18
TP - Fonctions de représentation	19
Contexte	19
Objectif	19
Fichiers fournis	19
Travail demandé	19
Étapes	19
Tests	19
Livrable	19
Critères de réussite	20
Aides progressives	20
Erreurs fréquentes	20
Auto-évaluation	20
Extension experte	20
Fiche méthode - Choisir une représentation	21
Situation-problème	21
Méthode	21
Exemple	21
Erreurs fréquentes	21
Auto-évaluation	21
Vérification finale	21
Aides progressives - S01	22
Situation-problème	22
Aides progressives	22
Erreurs fréquentes	22
Extension	22
Auto-évaluation	22
Différenciation	23
Renforcement après correction	23
Évaluation - S01	24
Durée	24
Matériel autorisé	24
Compétences évaluées	24
Barème	24
Questions progressives	24
Programmation	24
Justification	24
Analyse de code	24
Corrigé lié	25
Erreurs fréquentes	25
Auto-évaluation	25
Extension	25

S01 — Représenter des données : bits, textes et types construits

Situation-problème

Imaginons un badge de cantine identifié B-17. Ce badge contient un identifiant numérique (17), une variation de solde qui peut être négative (-3), un droit d'accès qui est soit autorisé soit refusé (vrai ou faux), et une initiale affichée sur l'écran (le caractère É). La machine qui lit ce badge ne manipule pourtant aucune de ces informations sous la forme que nous, humains, utilisons au quotidien : elle ne travaille qu'avec des suites de 0 et de 1, appelées **bits**.

Pour que la machine puisse stocker, traiter et restituer chacune de ces informations, il faut une **convention de représentation** — un accord sur la manière dont les bits correspondent à une valeur. Sans cette convention, la suite 11111101 pourrait aussi bien désigner le nombre 253 qu'un entier relatif valant -3 : les bits sont les mêmes, mais leur interprétation change tout. C'est cette question centrale — comment choisir une représentation adaptée à la donnée et au traitement attendu — que nous allons explorer dans cette séquence.

Questions de départ :

1. Parmi les quatre informations du badge (identifiant 17, variation -3 , accès autorisé, initiale É), lesquelles peut-on coder directement par un entier positif ?
2. Lesquelles nécessitent une convention supplémentaire (signe, valeur logique, encodage de caractère) ?
3. Si l'on lit la suite 11111101 comme un entier positif, on obtient 253. Mais si on la lit en complément à deux sur 8 bits, on obtient -3 . Qu'est-ce qui détermine la bonne lecture ?
4. Pourquoi le caractère É peut-il poser problème dans certains anciens systèmes d'encodage ?

Conclusion attendue : une suite de bits n'a de sens que si l'on connaît la convention de représentation utilisée.

Objectifs

À l'issue de cette séquence, vous saurez :

- définir ce qu'est un bit et expliquer son rôle comme unité élémentaire d'information ;
- convertir un entier positif entre les bases 2, 10 et 16, et repérer l'intervalle représentable sur un nombre de bits donné ;
- encoder et décoder de petits entiers relatifs en complément à deux ;
- construire la table de vérité d'une expression booléenne ;
- expliquer la différence entre un caractère, un point de code et un encodage ;
- choisir entre liste, tuple et dictionnaire en justifiant par les opérations attendues ;
- vérifier une fonction à l'aide de tests simples couvrant des cas ordinaires et des cas limites.

Prérequis

Cette séquence suppose que vous savez additionner et diviser des entiers, lire une écriture décimale positionnelle, exécuter une fonction Python simple, distinguer une valeur d'une variable et d'une expression, lire une condition `if`, et utiliser une liste Python en lecture.

Limites de la séquence

Cette séquence pilote regroupe deux blocs liés du programme de Première : les types et valeurs de base d'une part, les types construits d'autre part. Elle ne traite ni l'import de fichiers CSV, ni le tri ou la fusion de tables, qui relèvent d'une autre séquence. Les nombres flottants sont seulement signalés comme un sujet à part, car leur représentation (norme IEEE 754) mérite un traitement séparé. L'objectif ici est de consolider les représentations indispensables avant d'aborder les traitements de données tabulaires.

Formalisation

Une donnée informatique n'est pas seulement une valeur visible à l'écran : c'est une valeur **associée à une représentation**. La même suite de bits peut représenter des choses très différentes selon la convention adoptée. Par exemple, les huit bits 11111111 représentent 255 si l'on choisit la convention « entier positif sur 8 bits », mais -1 si l'on choisit la convention « complément à deux sur 8 bits ».

Choisir une représentation revient à répondre à trois questions fondamentales :

- **Quelles valeurs** doit-on pouvoir représenter ? (un entier positif ? un entier relatif ? un caractère ? une collection ?)
- **Quelles opérations** effectue-t-on le plus souvent ? (comparer ? additionner ? chercher par clé ? parcourir dans l'ordre ?)
- **Quelles erreurs** faut-il éviter ? (débordement de capacité ? confusion entre caractère et octet ? accès hors bornes ?)

Un bon choix de représentation simplifie les traitements et réduit les risques d'erreur. Un mauvais choix impose des conversions inutiles ou introduit des erreurs silencieuses.

Définitions

- **Bit** : unité élémentaire d'information, qui prend la valeur 0 ou 1.
- **Base de numération** : système positionnel utilisant un nombre fixé de chiffres (2 en binaire, 10 en décimal, 16 en hexadécimal).
- **Complément à deux** : convention de codage des entiers relatifs sur un nombre fixe de bits, où le bit de poids fort indique le signe.
- **Booléen** : valeur logique qui vaut True (vrai) ou False (faux).
- **Table de vérité** : tableau qui énumère toutes les combinaisons possibles des variables booléennes d'une expression et le résultat associé.
- **Encodage de texte** : règle qui associe des caractères à des suites d'octets (ASCII, UTF-8, etc.).
- **Liste Python** : collection ordonnée et modifiable, où les éléments sont repérés par leur index.
- **Tuple Python** : collection ordonnée, généralement utilisée comme p-uplet de valeurs liées, non modifié après création.
- **Dictionnaire Python** : structure qui associe des clés uniques à des valeurs, permettant un accès direct par clé.

Bases 2, 10 et 16

En base 10, nous utilisons dix chiffres (de 0 à 9) et chaque position correspond à une puissance de 10. En base 2 (binaire), seuls deux chiffres sont disponibles (0 et 1) et chaque position correspond

à une puissance de 2. En base 16 (hexadécimal), on dispose de seize symboles : les chiffres 0 à 9 complétés par les lettres A à F qui représentent respectivement les valeurs 10 à 15.

Dans toute écriture positionnelle, le **poïds** d'un chiffre dépend de sa position. Ainsi, l'écriture 1011_2 se décompose en $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$, soit $8 + 0 + 2 + 1 = 11_{10}$. De même, $2A_{16}$ se lit $2 \times 16^1 + 10 \times 16^0 = 32 + 10 = 42_{10}$.

La base 16 est particulièrement commode en informatique car chaque chiffre hexadécimal correspond exactement à 4 bits. Par exemple, $F_{16} = 1111_2$ et $A_{16} = 1010_2$. Un **octet** (8 bits) se représente donc toujours avec exactement deux chiffres hexadécimaux : $11111111_2 = FF_{16} = 255_{10}$.

Exemple corrigé 1 — Convertir 45_{10} en base 2

Pour convertir un entier positif de la base 10 vers la base 2, on effectue des divisions successives par 2 en notant chaque reste, puis on lit les restes du dernier au premier :

Division	Quotient	Reste
$45 \div 2$	22	1
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

En lisant les restes de bas en haut, on obtient : $45_{10} = 101101_2$. On vérifie : $32 + 8 + 4 + 1 = 45$. 

Contre-exemple 1 — « 101 vaut toujours cent-un »

C'est faux. L'écriture 101 dépend entièrement de la base dans laquelle on la lit : en base 10, elle vaut cent-un ; en base 2, 101_2 vaut cinq ($4 + 0 + 1$) ; en base 16, 101_{16} vaut deux-cent-cinquante-sept ($256 + 0 + 1$). C'est pourquoi il faut **toujours préciser la base** quand il y a ambiguïté.

Erreur fréquente 1 — Confondre nombre de chiffres décimaux et nombre de bits

Le nombre 100 s'écrit avec trois chiffres en base 10, mais sa représentation binaire 1100100_2 utilise sept bits. Le nombre de symboles nécessaires dépend de la base : plus la base est petite, plus il faut de symboles pour représenter la même valeur.

Exercice intégré 1

1. Convertir 31_{10} en base 2 par divisions successives.
2. Convertir 11110_2 en base 10 en développant les puissances de 2.
3. Convertir $2F_{16}$ en base 10.
4. Vérifier que 1111_2 et F_{16} représentent bien la même valeur.

Entiers positifs et intervalle représentable

Lorsqu'on dispose de n bits pour représenter un entier positif, les valeurs possibles vont de 0 à $2^n - 1$. Par exemple, sur 4 bits, l'intervalle est $[0 ; 15]$; sur 8 bits, il est $[0 ; 255]$; sur 16 bits, il atteint $[0 ; 65\,535]$. Le nombre de bits impose donc une **limite supérieure** : si un compteur codé sur 8 bits dépasse 255, la représentation déborde et le résultat devient incohérent.

Python masque en partie cette limite pour ses entiers natifs, car il utilise en interne une taille variable qui s'ajuste automatiquement. Cependant, dans les fichiers, les protocoles réseau, les formats d'image et les systèmes embarqués, les tailles sont fixées et la question du débordement est concrète.

Exemple corrigé 2 — Combien de bits pour écrire 100_{10} ?

On cherche le plus petit entier n tel que $2^n > 100$. Comme $2^6 = 64$ ne suffit pas et $2^7 = 128$ dépasse 100, il faut au minimum **7 bits**. En effet, $100_{10} = 1100100_2$, qui comporte bien 7 chiffres binaires.

Entiers relatifs et complément à deux

Pour représenter des entiers négatifs, il ne suffit pas d'ajouter un bit de signe devant un entier positif (cette convention, dite « signe-magnitude », crée deux représentations pour zéro et complique l'addition). La convention la plus utilisée est le **complément à deux**, qui présente l'avantage de permettre les additions et soustractions avec le même circuit matériel.

Sur n bits en complément à deux, l'intervalle représentable va de -2^{n-1} à $2^{n-1} - 1$. Concrètement :

- Sur 4 bits : de -8 à 7 .
- Sur 8 bits : de -128 à 127 .

Le **bit de poids fort** (le bit le plus à gauche) joue le rôle d'indicateur de signe : s'il vaut 0, le nombre est positif ou nul ; s'il vaut 1, le nombre est négatif.

Pour encoder un entier négatif $-x$ sur n bits, on calcule $2^n - x$. Par exemple, pour coder -5 sur 8 bits : $256 - 5 = 251$, et $251_{10} = 11111011_2$.

Pour décoder une valeur binaire dont le bit de poids fort vaut 1, on interprète d'abord la suite comme un entier positif, puis on soustrait 2^n . Par exemple, $11111011_2 = 251_{10}$, et comme le bit de gauche vaut 1, on calcule $251 - 256 = -5$.

Exemple corrigé 3 — Coder -5 en complément à deux sur 8 bits

1. On calcule $2^8 - 5 = 256 - 5 = 251$.
2. On convertit 251 en binaire : $251_{10} = 11111011_2$.
3. Le résultat est 11111011 (8 bits, bit de gauche à 1 : bien négatif).
4. Vérification par décodage : $11111011_2 = 251_{10}$, puis $251 - 256 = -5$. ☑

Contre-exemple 2 — La même suite de bits, deux valeurs différentes

La suite 11111011 représente **251** si on la lit comme un entier positif, mais **-5** si on la lit en complément à deux sur 8 bits. Ce n'est pas une erreur de calcul : c'est simplement que la convention d'interprétation n'est pas la même. Sans préciser cette convention, la suite de bits est ambiguë.

Exercice intégré 2

1. Sur 8 bits, encoder 7, -1 et -8 en complément à deux.
2. Sur 8 bits, décoder les suites 00001111, 11111111 et 11111000.
3. Donner l'intervalle représentable sur 4 bits en complément à deux.
4. Expliquer pourquoi la valeur 140 ne peut pas être codée comme entier relatif sur 8 bits (alors qu'elle entre sur 8 bits comme entier positif).

Booléens et tables de vérité

Un **booléen** ne peut prendre que deux valeurs : True (vrai) ou False (faux). En Python, les trois opérateurs fondamentaux sont and (et logique), or (ou logique) et not (négation). On peut combiner ces opérateurs pour former des **expressions booléennes** arbitrairement complexes.

Une **table de vérité** est un outil systématique qui liste toutes les combinaisons possibles des variables d'entrée et le résultat de l'expression pour chacune. Avec une variable, il y a 2 lignes ; avec deux variables, 4 lignes ; avec trois variables, 8 lignes (en général, 2^n lignes pour n variables).

Voici les règles de base à retenir :

- and vaut True uniquement si **les deux** entrées sont vraies ;
- or vaut True si **au moins une** entrée est vraie (c'est un « ou inclusif ») ;
- not inverse la valeur : not True donne False, et réciproquement.

Le **ou exclusif** (souvent noté XOR) est une opération différente de or : il vaut True si **exactement une** des deux entrées est vraie, mais False si les deux sont vraies. En Python, il n'existe pas de mot-clé xor pour les booléens, mais on peut l'exprimer par $(a \text{ and not } b) \text{ or } (\text{not } a \text{ and } b)$.

Exemple corrigé 4 — Table de vérité de $a \text{ and not } b$

a	b	not b	$a \text{ and not } b$
False	False	True	False
False	True	False	False
True	False	True	True
True	True	False	False

Seule la combinaison $a = \text{True}$, $b = \text{False}$ donne True. La table de vérité permet de raisonner de manière exhaustive au lieu de deviner le résultat.

Erreur fréquente 2 — Confondre or et « ou exclusif »

Beaucoup d'élèves croient que or signifie « l'un ou l'autre, mais pas les deux ». C'est faux en Python : True or True vaut True, car or est **inclusif**. Si l'on veut un ou exclusif, il faut le formuler explicitement, par exemple avec $(a \text{ and not } b) \text{ or } (\text{not } a \text{ and } b)$.

Exercice intégré 3

1. Dresser la table de vérité complète de not a or b (4 lignes).
2. Dresser la table de vérité de $(a \text{ and } b) \text{ or } (a \text{ and not } b)$.

3. Observer le résultat de la deuxième table : peut-on simplifier cette expression ?

Texte, ASCII et Unicode

Un texte informatique est une suite de **caractères**, mais un caractère n'est pas directement un octet. Pour passer de l'un à l'autre, il faut un **système d'encodage** qui définit comment chaque caractère est représenté en mémoire.

ASCII (American Standard Code for Information Interchange) est un ancien encodage qui attribue un code numérique à 128 caractères : les lettres anglaises non accentuées, les chiffres, quelques signes de ponctuation et des caractères de contrôle. Par exemple, la lettre A a le code 65 et la lettre a le code 97. Cet encodage suffit pour l'anglais de base, mais il est incapable de représenter les lettres accentuées du français, les caractères arabes, chinois, ou les émojis.

Unicode résout ce problème en attribuant un **point de code** unique à un très grand nombre de caractères (plus de 150 000 aujourd'hui). Le point de code est un numéro abstrait : par exemple, A a le point de code U+0041 (soit 65 en décimal) et é a le point de code U+00E9 (soit 233). Mais le point de code ne dit pas encore comment le caractère est stocké en octets.

UTF-8 est l'encodage le plus courant d'Unicode en octets. Il utilise un nombre variable d'octets par caractère : un seul octet pour les caractères ASCII (compatibilité totale), deux octets pour les lettres accentuées européennes, et jusqu'à quatre octets pour les caractères rares ou les émojis. C'est pourquoi le nombre de caractères d'une chaîne n'est pas toujours égal au nombre d'octets qu'elle occupe.

Exemple corrigé 5 — Points de code et encodage UTF-8

En Python, la fonction `ord` renvoie le point de code d'un caractère, et `chr` fait l'inverse :

```
>>> ord("A")
65
>>> chr(65)
'A'
>>> ord("é")
233
```

Pour voir la représentation en octets, on peut utiliser la méthode `encode` :

```
>>> "A".encode("utf-8")
b'\x41'          # 1 octet
>>> "é".encode("utf-8")
b'\xc3\xa9'      # 2 octets
```

La lettre A occupe un seul octet (car son point de code est inférieur à 128), tandis que é en occupe deux. Cela signifie que pour une chaîne comme "Aé", la longueur en caractères est 2, mais la taille en octets UTF-8 est 3.

Contre-exemple 3 — « Un caractère vaut toujours un octet »

Cette affirmation est fausse en général. Elle est vraie uniquement pour les 128 caractères ASCII de base. Dès qu'on utilise un caractère accentué, un idéogramme ou un émoji, l'encodage UTF-8

nécessite plusieurs octets. Confondre le nombre de caractères et le nombre d'octets peut provoquer des erreurs dans le traitement de fichiers texte ou dans les communications réseau.

Listes et tableaux Python

En Python, une **liste** est une collection ordonnée et modifiable d'éléments, repérés par des **index** entiers commençant à 0. On peut lire un élément (`notes[0]`), le modifier (`notes[1] = 14`), ajouter un élément en fin de liste (`notes.append(18)`), ou parcourir tous les éléments avec une boucle `for`.

La liste est le type construit le plus polyvalent en Python. Elle convient bien lorsque l'**ordre** des éléments compte, lorsqu'on a besoin de **parcourir** tous les éléments, ou lorsqu'on veut **modifier** la collection au fil du programme.

Une **compréhension de liste** permet de construire une liste à partir d'une règle concise. Par exemple, `[x*x for x in range(5)]` produit la liste `[0, 1, 4, 9, 16]` en élevant au carré chaque entier de 0 à 4.

On peut aussi représenter une **matrice** (tableau à deux dimensions) par une liste de listes. Par exemple, si `m = [[1, 2, 3], [4, 5, 6]]`, alors `m[1][2]` désigne l'élément à la ligne 1, colonne 2, soit la valeur 6.

Exemple corrigé 6 — Lecture, modification et compréhension

```
valeurs = [4, 7, 9]
```

- `valeurs[0]` vaut 4 (premier élément, index 0).
- `valeurs[1]` vaut 7 (deuxième élément, index 1).
- `valeurs[2]` vaut 9 (troisième élément, index 2).
- La somme se calcule par `total = sum(valeurs)`, soit 20.
- La compréhension `[v + 1 for v in valeurs]` produit `[5, 8, 10]` : chaque élément est augmenté de 1.

Erreur fréquente 3 — Index commençant à 1

En Python, les index commencent à 0, pas à 1. Dans une liste de longueur 3, les index valides sont 0, 1 et 2. Tenter d'accéder à `valeurs[3]` provoque une erreur `IndexError`. C'est un cas limite classique qu'il faut systématiquement tester.

Exercice intégré 4

1. Écrire une compréhension de liste qui double chaque élément de `[3, 5, 8]`.
2. Soit `m = [[1, 0], [0, 1]]`. Que vaut `m[0][1]` ? Et `m[1][0]` ?
3. Écrire une boucle qui additionne tous les éléments d'une liste `nombre`.

Tuples et p-uplets

Un **tuple** Python est une collection ordonnée qui sert à regrouper un petit nombre de valeurs liées entre elles. Contrairement à une liste, un tuple n'est généralement pas modifié après sa création (on dit qu'il est **immutable** dans l'usage courant, bien que Python n'interdise pas de remplacer la variable qui le contient).

Le tuple est adapté aux situations où le **nombre de champs est fixe** et où l'**ordre des champs est connu** par convention. Par exemple, un point du plan se représente naturellement par (x, y), et une couleur RGB par (rouge, vert, bleu). En revanche, un tuple devient fragile et peu lisible dès que le nombre de champs augmente, car le lecteur doit mémoriser la position de chaque valeur.

L'intérêt principal du tuple dans le cadre du programme est qu'une **fonction peut renvoyer un p-uplet** — c'est-à-dire plusieurs valeurs regroupées en un seul objet retourné.

Exemple corrigé 7 — Fonction renvoyant un tuple (milieu de deux points)

On veut écrire une fonction qui calcule le milieu de deux points du plan, chaque point étant représenté par un tuple (x, y) :

```
def milieu(p1, p2):  
    return ((p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2)
```

L'appel `milieu((1, 3), (5, 7))` renvoie le tuple (3.0, 5.0). Le tuple permet de renvoyer les deux coordonnées du milieu en un seul objet, sans avoir à créer deux variables séparées ou à utiliser une structure plus lourde.

Contre-exemple 4 — Un tuple trop long perd en lisibilité

Représenter une fiche élève par un tuple comme (id, nom, groupe, option, date, mail) est techniquement possible, mais fragile : le lecteur du code doit se souvenir que l'index 3 correspond à l'option et l'index 5 au mail. Un **dictionnaire** avec des clés nommées (`{"id": 42, "nom": "Ada", ...}`) serait ici bien plus lisible et moins sujet aux erreurs d'index.

Dictionnaires

Un **dictionnaire** Python associe des **clés** à des **valeurs**. Chaque clé est unique et permet d'accéder directement à la valeur correspondante, sans avoir à parcourir toute la structure. Par exemple, `stock = {"stylo": 18, "cahier": 5}` associe la clé "stylo" à la valeur 18 et la clé "cahier" à la valeur 5. On accède ensuite au stock de stylos par `stock["stylo"]`.

Le dictionnaire est le type construit adapté lorsque l'opération la plus fréquente est un **accès par nom ou identifiant**, lorsqu'on veut **compter des occurrences** (chaque clé étant un élément à compter, et la valeur associée étant l'effectif), ou lorsqu'on veut représenter un **enregistrement** avec des champs nommés.

Les méthodes `keys()`, `values()` et `items()` permettent de parcourir un dictionnaire :

- `stock.keys()` donne les clés : "stylo", "cahier".
- `stock.values()` donne les valeurs : 18, 5.
- `stock.items()` donne les couples clé-valeur : ("stylo", 18), ("cahier", 5).

Exemple corrigé 8 — Compter des votes avec un dictionnaire

On dispose d'une liste de votes : `votes = ["A", "B", "A", "C", "A"]`. On veut compter le nombre de votes pour chaque choix.

```
effectifs = {}  
for choix in votes:  
    if choix in effectifs:  
        effectifs[choix] += 1  
    else:  
        effectifs[choix] = 1
```

Après exécution, `effectifs` vaut `{"A": 3, "B": 1, "C": 1}`. La clé est le choix de vote, la valeur est l'effectif. Le dictionnaire est ici la structure adaptée car on a besoin d'associer chaque choix à son compte, et d'y accéder directement par le nom du choix.

Contre-exemple 5 — Liste de couples vs dictionnaire

On pourrait stocker les mêmes données avec une liste de couples : `[("A", 3), ("B", 1), ("C", 1)]`. Mais si l'on cherche souvent le nombre de votes pour un choix donné (par exemple "C"), il faudrait parcourir toute la liste à chaque fois, car les éléments ne sont accessibles que par leur position. Avec un dictionnaire, l'accès par clé `effectifs["C"]` est direct. Le choix entre les deux structures dépend donc des opérations que l'on prévoit d'effectuer le plus souvent.

Choisir une représentation

Le choix entre liste, tuple et dictionnaire ne se fait pas au hasard : il doit être guidé par les **opérations prévues** sur les données.

Situation	Structure adaptée	Justification
Suite de mesures à parcourir et modifier	Liste	L'ordre compte, les valeurs changent
Coordonnées (x, y) d'un point fixe	Tuple	Nombre de champs fixe, pas de modification
Stock de produits accessibles par référence	Dictionnaire	L'accès se fait par nom de produit
Table de pixels ligne par ligne	Liste de listes	Structure à deux dimensions, modifiable
Fiche courte avec champs nommés	Dictionnaire	L'accès par nom de champ est prioritaire

Pour les types de base, le choix de la taille en bits détermine l'intervalle représentable (entiers positifs ou relatifs), et le choix de l'encodage détermine comment les caractères sont stockés. Dans tous les cas, le choix doit être **justifié par un traitement** — c'est-à-dire par une opération concrète que l'on effectuera sur les données.

Tests et cas limites

Un **test** vérifie qu'une fonction produit le résultat attendu pour une entrée choisie. Un **cas limite** est une valeur particulière qui risque de révéler une erreur de programmation : une valeur à la frontière du domaine de validité, un cas dégénéré, ou une entrée qui déclenche un traitement spécial.

Voici des exemples de cas limites pertinents pour cette séquence :

- **Conversions de base** : tester avec 0 (traitement spécial dans beaucoup d'algorithmes), avec 1 (plus petit entier non nul), et avec la plus grande valeur autorisée par le nombre de bits.
- **Complément à deux** : tester -1 (tous les bits à 1), les bornes de l'intervalle (-128 et 127 sur 8 bits), et une valeur juste hors de l'intervalle.
- **Texte et encodage** : tester avec un caractère accentué ("é") et un caractère multi-octets.
- **Listes** : tester avec une liste vide ([]).
- **Dictionnaires** : tester l'accès à une clé absente (provoque un `KeyError`).

Un jeu de tests ne prouve jamais que la fonction est correcte dans tous les cas possibles, mais un bon jeu de tests — couvrant des cas ordinaires **et** des cas limites — réduit considérablement le risque d'erreurs non détectées.

Le fichier Python associé à ce cours est `python/representation_tools.py`, et les tests correspondants sont dans `tests/test_representation_tools.py`. Vous pouvez les exécuter pour vérifier le bon fonctionnement de chaque fonction.

Erreurs fréquentes — Récapitulatif

1. **Oublier de préciser la base** d'une écriture : 101 n'a pas de sens unique sans indication de base.
2. **Lire un complément à deux comme un entier positif** : 11111111 ne vaut pas 255 si la convention est le complément à deux sur 8 bits.
3. **Confondre caractère et octet** : "é" est un seul caractère mais occupe deux octets en UTF-8.
4. **Utiliser une liste quand un dictionnaire s'impose** : si l'accès par clé est fréquent, une liste force un parcours inutile.
5. **Oublier que les index Python commencent à zéro** : dans une liste de 3 éléments, le dernier index est 2, pas 3.
6. **Croire qu'un test réussi prouve tout** : un test ne couvre qu'un cas ; il faut varier les entrées et inclure des cas limites.

À retenir

- Une suite de bits n'a de sens que par la convention de représentation qui lui est associée.
- Les bases 2, 10 et 16 sont trois écritures positionnelles d'une même valeur entière.
- Le complément à deux permet de coder les entiers relatifs sur un nombre fixé de bits, avec un intervalle asymétrique (une valeur négative de plus que de valeurs positives).
- Une table de vérité rend explicite le comportement d'une expression booléenne pour toutes les entrées possibles.
- Unicode attribue un point de code à chaque caractère ; UTF-8 est l'encodage le plus courant pour stocker ces points de code en octets.
- Une liste est ordonnée et modifiable ; un tuple regroupe un petit nombre de valeurs liées ; un dictionnaire associe des clés à des valeurs.
- Le choix d'une représentation dépend toujours des traitements que l'on prévoit d'effectuer.
- Les tests doivent couvrir des cas ordinaires **et** des cas limites pour être utiles.

Auto-évaluation

Avant de passer au TD, vérifiez que vous savez :

- convertir 37_{10} en base 2 et $2A_{16}$ en base 10 ;
- donner l'intervalle des entiers relatifs représentables sur 8 bits ;
- décoder 11111110 en complément à deux sur 8 bits ;
- dresser la table de vérité d'une expression à deux variables ;
- expliquer pourquoi é n'est pas un caractère ASCII simple ;
- choisir entre liste, tuple et dictionnaire pour un problème donné ;
- proposer au moins deux cas limites pour tester une fonction de conversion.

Trace écrite - S01

Notions essentielles

- Une donnée est une valeur accompagnée d'une convention de représentation.
- Le bit est l'unité minimale : 0 ou 1.
- La base 2 utilise deux chiffres, la base 10 dix chiffres, la base 16 seize chiffres.
- Un octet contient 8 bits.
- Deux chiffres hexadécimaux représentent un octet.
- Un entier positif sur n bits va de 0 à $2^n - 1$.
- Un entier relatif en complément à deux sur n bits va de $-2^{(n-1)}$ à $2^{(n-1)} - 1$.
- Un booléen vaut True ou False.
- Une table de vérité énumère tous les cas d'une expression booléenne.
- Un texte nécessite un encodage.
- ASCII est limité ; Unicode vise une représentation beaucoup plus large.
- Une liste est ordonnée et modifiable.
- Un tuple regroupe un nombre fixé de valeurs.
- Un dictionnaire associe clés et valeurs.

Définitions

- Bit : unité d'information binaire.
- Base : système positionnel déterminé par un nombre de chiffres.
- Complément à deux : codage des entiers relatifs sur un nombre fixé de bits.
- Booléen : valeur logique vraie ou fausse.
- Encodage : règle qui transforme un caractère en octets.
- Liste : collection indexée et modifiable.
- Tuple : p-uplet ordonné.
- Dictionnaire : association clé-valeur.

Méthodes

- Pour convertir une base vers 10, développer avec les puissances de la base.
- Pour convertir de 10 vers une base, diviser successivement par la base.
- Pour lire un complément à deux, regarder le bit de gauche.
- Si le bit de gauche vaut 1, soustraire 2^n à la valeur non signée.
- Pour construire une table de vérité, lister tous les couples ou triplets de booléens.
- Pour choisir une structure, identifier l'opération principale.
- Accès par position : liste ou tuple.
- Accès par nom ou identifiant : dictionnaire.
- Données fixes et groupées : tuple.
- Données modifiables et ordonnées : liste.

Exemples minimaux

- $1011_2 = 11_{10}$.
- $2A_{16} = 42_{10}$.
- Sur 8 bits, 11111111 vaut -1 en complément à deux.

- True and False vaut False.
- ord("A") vaut 65.
- (3, 4) peut représenter un point.
- [3, 4] peut représenter une liste modifiable.
- {"A": 3} associe la clé "A" à la valeur 3.

Points de vigilance

- Ne pas lire une écriture sans connaître sa base.
- Ne pas lire un complément à deux comme un entier positif.
- Ne pas confondre caractère et octet.
- Ne pas commencer les index Python à 1.
- Ne pas utiliser une liste quand l'accès par clé domine.
- Ne pas croire qu'un jeu de tests prouve tous les cas.

À savoir refaire

- Convertir un entier positif entre bases 2, 10 et 16.
- Trouver l'intervalle des relatifs sur n bits.
- Encoder et décoder un petit relatif sur 8 bits.
- Construire la table de vérité d'une expression à deux variables.
- Donner les points de code d'une chaîne courte avec ord.
- Lire et modifier un élément de liste.
- Ecrire une fonction qui renvoie un tuple.
- Parcourir les couples clé, valeur d'un dictionnaire.
- Proposer des tests ordinaires et des cas limites.

Auto-positionnement

- Je convertis 19_{10} en base 2 sans aide.
- Je convertis $1F_{16}$ en base 10 sans aide.
- J'explique pourquoi 11111110 peut valoir 254 ou -2.
- Je dresse une table de vérité à deux variables.
- Je donne un exemple où un dictionnaire est préférable à une liste.
- Je propose un test pour to_base(0, 2).

TD - S01 Représentation des données

Situation-problème

Un système d'inscription stocke des identifiants, des droits d'accès, des textes et des réponses à un questionnaire. Chaque information doit être représentée sans ambiguïté. Le TD entraîne à choisir et manipuler ces représentations.

Objectifs

- Convertir sans ambiguïté entre bases.
- Décoder un complément à deux.
- Construire une table de vérité.
- Choisir une structure Python.
- Analyser du code et proposer des tests.

Exercices

Exercice 1 - Socle - Bases

Convertir 18_{10} , 31_{10} , 42_{10} en base 2. Convertir 1010_2 , 1111_2 , 100000_2 en base 10. Convertir $2A_{16}$ et FF_{16} en base 10. Justifier chaque conversion avec les puissances utilisées.

Exercice 2 - Socle - Octets

Donner la plus petite et la plus grande valeur représentable par un entier positif sur 8 bits. Dire si 255, 256, 128, 0 peuvent être représentés sur 8 bits sans signe. Ecrire 255 en base 2 et en base 16. Expliquer pourquoi deux chiffres hexadécimaux suffisent pour un octet.

Exercice 3 - Standard - Complément à deux

Sur 8 bits, encoder 5, -1, -7, -128. Sur 8 bits, décoder 00000101, 11111111, 11111001, 10000000. Donner l'intervalle représentable sur 4 bits. Dire pourquoi -9 ne se code pas sur 4 bits.

Exercice 4 - Socle - Booléens

Dresser la table de vérité de $a \text{ and } b$. Dresser la table de vérité de $a \text{ or } b$. Dresser la table de vérité de $a \text{ and not } b$. Comparer or inclusif et xor.

Exercice 5 - Standard - Texte

Avec Python, on obtient `ord("A") = 65` et `ord("é") = 233`. Expliquer ce que représente ce nombre. Comparer `"A".encode("utf-8")` et `"é".encode("utf-8")`. Dire pourquoi compter les caractères ne suffit pas toujours pour connaître le nombre d'octets.

Exercice 6 - Socle - Tuples et listes

On représente un point par (x, y) . Ecrire une fonction `milieu(p1, p2)` qui renvoie le point milieu. On représente des températures par une liste. Ecrire une compréhension qui ajoute 1 à

chaque température. Justifier pourquoi le point est mieux représenté par un tuple que par deux variables isolées.

Exercice 7 - Standard - Dictionnaires

On dispose de `votes = ["A", "B", "A", "C", "A", "B"]`. Construire le dictionnaire des effectifs. Identifier la clé de plus grand effectif. Expliquer pourquoi un dictionnaire est adapté à cette tâche. Donner un cas où une liste serait suffisante.

Exercice 8 - Expert - Choix de représentation

Pour chaque situation, choisir liste, tuple ou dictionnaire. Situation A : positions successives d'un robot. Situation B : coordonnées fixes d'un point. Situation C : stock par référence produit. Situation D : table de pixels ligne par ligne. Situation E : fiche courte avec champs nommés. Justifier chaque choix par les opérations prévues.

Exercice 9 - Analyse de code

Lire la fonction suivante. Elle doit convertir un entier positif en base 2.

```
def mystere(n):  
    bits = ""  
    while n > 0:  
        bits = str(n % 2) + bits  
        n = n // 2  
    return bits
```

Tester mentalement `mystere(6)`. Identifier le cas limite non géré. Proposer une correction. Proposer trois tests.

Exercice 10 - Expert - Justification

Un camarade stocke des utilisateurs sous forme de liste de tuples (`identifiant`, `nom`). Le programme cherche très souvent un nom à partir d'un identifiant. Expliquer le problème. Proposer une représentation plus adaptée. Donner un exemple de transformation vers cette représentation. Donner un test qui vérifie la transformation.

Aides progressives

- Niveau 1 : écrire d'abord les puissances de la base.
- Niveau 2 : pour le complément à deux, noter le nombre de bits avant de calculer.
- Niveau 3 : pour les structures Python, écrire l'opération la plus fréquente.

Erreurs fréquentes

- Lire 11111111 sans préciser la convention.
- Confondre or et xor.
- Oublier le cas `n = 0` dans une conversion.
- Choisir une liste alors qu'une clé est utilisée partout.

Auto-évaluation

- Je sais convertir dans les deux sens.
- Je sais produire une table de vérité.
- Je sais expliquer un choix de structure.
- Je sais proposer des tests pertinents.

Corrigé associé

Les réponses détaillées sont dans `corrige.md`, sections TD.

Écriture de code

Les exercices 6, 7 et 9 demandent une écriture de code Python.

Le code attendu doit être court, testé sur un cas normal et testé sur un cas limite.

La correction associée précise les variantes acceptables.

TP - Fonctions de représentation

Contexte

Le cours a posé les conventions. Le TP transforme ces conventions en fonctions testables. Le fichier support est `python/representation_tools.py`. Les tests de référence sont dans `tests/test_representation_tools.py`.

Objectif

Implémenter, lire et tester des fonctions qui convertissent des entiers, manipulent le complément à deux et choisissent une structure Python.

Fichiers fournis

- `python/representation_tools.py`
- `tests/test_representation_tools.py`

Travail demandé

Lire chaque fonction avant de l'exécuter. Associer chaque fonction à une capacité du cours. Compléter le tableau de suivi sur papier ou cahier. Tester `to_base` avec 0, 1, 42, 255. Tester `from_base` avec "0", "101010", "FF". Tester `encode_twos_complement` avec -1, -5, 127. Tester `decode_twos_complement` avec 11111111, 11111011, 01111111. Tester `unicode_codepoints` avec "A" et "Aé". Tester `choose_container` avec trois scénarios. Rédiger une phrase de justification pour chaque structure choisie.

Étapes

Étape 1 : ouvrir le fichier Python. Étape 2 : repérer les préconditions de `to_base`. Étape 3 : exécuter les tests existants. Étape 4 : ajouter sur brouillon un test pour une base invalide. Étape 5 : expliquer pourquoi 0 doit être traité à part. Étape 6 : lire le codage de -5 en complément à deux. Étape 7 : vérifier l'intervalle représentable sur 8 bits. Étape 8 : construire une table de vérité xor avec la fonction fournie. Étape 9 : comparer les points de code de "A" et "é". Étape 10 : choisir une structure pour un stock par référence. Étape 11 : choisir une structure pour une couleur RGB. Étape 12 : choisir une structure pour une suite de mesures.

Tests

Commande de référence : `python scripts/run_python_tests.py`. Un test doit contenir une entrée. Un test doit contenir un résultat attendu. Un test doit expliquer ce que l'on cherche à sécuriser. Test ordinaire : `to_base(42, 2) == "101010"`. Test limite : `to_base(0, 2) == "0"`. Test d'erreur : une base 1 doit provoquer une erreur. Test de signe : `decode_twos_complement("11111111") == -1`. Test Unicode : `unicode_codepoints("Aé") == [65, 233]`.

Livrable

Rendre un court fichier ou cahier contenant : - les résultats des tests ; - trois explications de choix de structure ; - un exemple de cas limite ; - une remarque sur une erreur évitée.

Critères de réussite

- Les tests fournis s'exécutent.
- Les conversions sont justifiées.
- Les bornes du complément à deux sont respectées.
- Les choix liste, tuple, dictionnaire sont argumentés.
- Les erreurs sont reliées à des cas limites.

Aides progressives

- Niveau 1 : lire la docstring avant le code.
- Niveau 2 : tester une seule fonction à la fois.
- Niveau 3 : écrire l'entrée, la sortie attendue et la raison du test.

Erreurs fréquentes

- Modifier le code avant d'avoir exécuté les tests.
- Confondre valeur Python et représentation textuelle.
- Tester seulement des cas ordinaires.
- Oublier les erreurs attendues.

Auto-évaluation

- Je sais relier une fonction à une capacité.
- Je sais exécuter les tests.
- Je sais expliquer un cas limite.
- Je sais proposer un test d'erreur.

Extension experte

Ajouter une fonction `intervalle_complement(bits)` qui renvoie le plus petit et le plus grand entier relatif représentable. Ajouter les tests pour `bits = 4` et `bits = 8`.

Fiche méthode - Choisir une représentation

Situation-problème

On doit stocker une information mais plusieurs représentations sont possibles. Le risque est de choisir la première représentation venue. La méthode impose de partir du traitement attendu.

Méthode

Étape 1 : nommer la donnée. Étape 2 : identifier les valeurs possibles. Étape 3 : repérer si la donnée peut être négative. Étape 4 : repérer si la donnée est logique, numérique, textuelle ou composée. Étape 5 : identifier l'opération la plus fréquente. Étape 6 : choisir une représentation candidate. Étape 7 : vérifier un cas ordinaire. Étape 8 : vérifier un cas limite. Étape 9 : rédiger la justification.

Exemple

Donnée : code de badge. Valeurs possibles : entiers positifs. Opération fréquente : comparaison d'identifiants. Représentation candidate : entier positif en base 10 côté humain, binaire côté machine. Cas limite : identifiant 0. Justification : l'identifiant ne porte pas de signe et doit être comparé rapidement.

Erreurs fréquentes

- Choisir une liste alors que l'accès se fait par clé.
- Oublier que le nombre de bits limite l'intervalle.
- Ecrire une conversion sans préciser la base.
- Tester seulement une valeur ordinaire.

Auto-évaluation

- Je sais identifier l'opération principale.
- Je sais citer un cas limite.
- Je sais justifier liste, tuple ou dictionnaire.
- Je sais expliquer le rôle de la convention de lecture.

Vérification finale

Je vérifie l'unité de représentation : bit, octet, caractère ou collection.

Je vérifie le domaine de valeurs accepté.

Je vérifie un cas limite avant de généraliser.

Je relie toujours la méthode au TD ou au TP.

Aides progressives - S01

Situation-problème

Un blocage en représentation vient souvent d'une convention oubliée. Les aides ci-dessous guident sans donner directement toute la réponse.

Aides progressives

Niveau 1 - Comprendre la question

- Entourer la base demandée.
- Souligner le nombre de bits.
- Repérer si la valeur peut être négative.
- Repérer si la donnée est une collection.
- Nommer l'opération principale : lire, modifier, chercher, compter, convertir.

Niveau 2 - Mettre en route

- Ecrire les puissances de 2 ou de 16.
- Pour la conversion vers base 10, multiplier chaque chiffre par son poids.
- Pour la conversion depuis base 10, faire les divisions successives.
- Pour le complément à deux, calculer l'intervalle représentable.
- Pour une table de vérité, lister tous les couples False/True.
- Pour une structure Python, essayer une phrase : "j'ai besoin de chercher par..."

Niveau 3 - Vérifier

- Refaire la conversion dans l'autre sens.
- Vérifier que le résultat tient dans le nombre de bits.
- Tester une valeur limite.
- Vérifier qu'une clé de dictionnaire existe avant de l'utiliser.
- Lire le résultat avec une phrase humaine.

Erreurs fréquentes

- Lire trop vite la base.
- Oublier la ligne False, False d'une table de vérité.
- Croire que 11111111 a une seule signification.
- Ecrire un test qui reproduit seulement l'exemple du cours.

Extension

Créer une mini-fiche personnelle : une erreur que je fais souvent, un exemple, un test qui la détecte.

Auto-évaluation

- Je sais choisir une aide adaptée.
- Je sais expliquer ce que l'aide m'a permis de débloquent.

- Je sais refaire sans aide après correction.

Différenciation

- Fragile : utiliser les aides niveau 1 et 2 avec nombres inférieurs à 32.
- Standard : utiliser les aides niveau 2 et 3 sur 8 bits.
- Expert : rédiger une aide supplémentaire pour un camarade.

Renforcement après correction

Reprendre une erreur en indiquant la représentation utilisée.

Dire si l'erreur porte sur la valeur, le codage ou le type Python.

Refaire un exemple analogue avec des nombres plus petits.

Comparer la réponse obtenue avec un calcul mental simple.

Expliquer à un camarade la première étape seulement.

Noter la question à poser au professeur si le blocage persiste.

Évaluation - S01

Durée

50 minutes.

Matériel autorisé

Aucun ordinateur. Calculatrice simple autorisée. Trace écrite non autorisée.

Compétences évaluées

- Convertir entre bases.
- Interpréter un complément à deux.
- Construire une table de vérité.
- Identifier un encodage de texte.
- Choisir une structure Python.
- Proposer un test.

Barème

Total : 20 points. Conversions : 4 points. Complément à deux : 4 points. Booléens et texte : 4 points. Structures Python : 5 points. Tests et justification : 3 points.

Questions progressives

Question 1 : convertir 29_{10} en base 2. Question 2 : convertir 101101_2 en base 10. Question 3 : convertir $3F_{16}$ en base 10. Question 4 : donner l'intervalle des entiers relatifs sur 8 bits. Question 5 : décoder 11111110 comme complément à deux sur 8 bits. Question 6 : coder -6 en complément à deux sur 8 bits. Question 7 : dresser la table de vérité de $a \text{ or } \text{not } b$. Question 8 : expliquer pourquoi un caractère Unicode peut occuper plusieurs octets.

Programmation

Question 9 : écrire une fonction `point(x, y)` qui renvoie un tuple. Question 10 : écrire une expression qui transforme `[1, 2, 3]` en `[2, 3, 4]`. Question 11 : construire un dictionnaire qui associe "rouge" à 3 et "bleu" à 2. Question 12 : choisir une représentation pour un stock par référence et justifier.

Justification

Chaque conversion doit afficher une méthode. Chaque choix de structure doit citer une opération. Chaque test doit indiquer entrée et résultat attendu.

Analyse de code

On donne :


```
def premier(liste):  
    return liste[0]
```

Question 13 : donner un cas qui fonctionne. Question 14 : donner un cas limite qui provoque une erreur. Question 15 : proposer un test pour ce cas limite.

Corrigé lié

Le corrigé détaillé est dans `corrige.md`.

Erreurs fréquentes

- Oublier la base.
- Oublier le nombre de bits.
- Donner une table de vérité incomplète.
- Proposer un test sans résultat attendu.

Auto-évaluation

- J'ai justifié mes conversions.
- J'ai précisé la convention de signe.
- J'ai écrit tous les cas booléens.
- J'ai relié chaque structure à une opération.

Extension

Question bonus non comptée dans le barème : expliquer pourquoi les flottants nécessitent une séquence dédiée.